

WikForge 核心源码精读笔记（源码 + 逐段解析）

用法：左边/上面是项目真实源码（已按学习需要删减次要行，删节处用 `# ...` 标注），下面紧跟解析。

覆盖四个核心模块：LLM 网关 → RAG 引擎 → 混合检索 → 入库流水线。

模块一：LLM 网关

backend/app/services/llm_gateway.py

全项目所有调 LLM 的地方（RAG 生成、查询改写、HyDE、文档解析）都经过它。底层是 LiteLLM，上面包了一层生产逻辑。

1.1 统一返回类型与错误类型

```
@dataclass
class LLMResponse:
    content: str = ""
    model: str = ""
    usage: dict = field(default_factory=dict)      # prompt_tokens / completion_tokens /
total_tokens
    finish_reason: str = "stop"

class LLMGatewayError(Exception):
    def __init__(self, message: str, reason: str = "unknown"):
        super().__init__(message)
        self.reason = reason      # timeout / rate_limit / auth / model_unavailable / unknown
```

解析：

- 项目**不依赖** LangChain 的 `AIMessage`，自定义 `LLMResponse`，下游服务只依赖项目自己的类型 → 框架可随时替换。
- `reason` 是机器可读错误码，上层据此把 429 映射成“请稍后再试”、把 timeout 映射成友好提示，不用解析各家 SDK 的异常文本。

1.2 模型名规范化 + drop_params（多厂商统一的核心）

```
import litellm

# 让 LiteLLM 自动丢弃上游模型不支持的参数（例如 gpt-5 系列只接受 temperature=1）
litellm.drop_params = True

# 给 model 自动加 openai/ 前缀，统一走 OpenAI 兼容协议
normalized_model = (
    effective_model if "/" in effective_model else f"openai/{effective_model}"
)

kwargs = {
    "model": normalized_model,      # "openai/gpt-4o" / "anthropic/claude-3-5-sonnet" ...
    "messages": messages,          # OpenAI Chat Completions 格式
```

```

        "temperature": temperature,
        "max_tokens": max_tokens,
        "timeout": self.timeout,
    }
    if self.api_base:
        kwargs["api_base"] = self.api_base
    if self.api_key:
        kwargs["api_key"] = self.api_key

    response = await litellm.acompletion(**kwargs)

```

解析：

- LiteLLM 用 `厂商/模型名` 前缀路由到对应厂商，并把请求/响应统一翻译成 OpenAI 格式。换模型只改环境变量 `LITELLM_MODEL`，不改代码——这就是“供应商解耦”。
- 对比你用的 `ChatOpenAI`：它只认 OpenAI 协议，换厂商要换类、装 partner 包。
- `drop_params = True` 是多模型共存的防御性写法：各厂商支持的参数集不同，硬传 `temperature=0.1` 给不支持的模型会直接报错，`drop_params` 让 LiteLLM 自动丢弃不兼容参数。

1.3 双层超时（SDK 超时 + asyncio 兜底）

```

response = await asyncio.wait_for(
    litellm.acompletion(**kwargs),  # kwargs 内已有 LiteLLM 自己的 timeout
    timeout=self.timeout + 5,       # 外层再加 5 秒保险
)

```

超时优先级（`_resolve_timeout`）：**构造函数显式传入** > `Settings.LLM_TIMEOUT`（默认 60s）> 模块常量 `DEFAULT_LLM_TIMEOUT`。

解析：

- 单层 SDK 超时的风险：SDK 内部重试/连接池卡死时，它自己的 timeout 可能不生效。外面再套 `asyncio.wait_for` 是“不信任下游”的防御式编程。
- `ChatOpenAI` 只有一个 timeout 参数透传，没有这层兜底。

1.4 错误分类翻译

```

except asyncio.TimeoutError:
    raise LLMGatewayError(f"LLM call timed out after {self.timeout} seconds",
        reason="timeout")
except Exception as e:
    error_msg = str(e)
    reason = "unknown"
    if "rate_limit" in error_msg.lower() or "429" in error_msg:
        reason = "rate_limit"
    elif "auth" in error_msg.lower() or "401" in error_msg or "403" in error_msg:
        reason = "auth"
    elif "not found" in error_msg.lower() or "404" in error_msg:
        reason = "model_unavailable"
    elif "timeout" in error_msg.lower():

```

```

reason = "timeout"
raise LLMGatewayError(f"LLM call failed: {error_msg}", reason=reason)

```

解析：把 100+ 厂商五花八门的异常归一化成 5 种业务错误码，上层 switch 处理即可。这是网关层最典型的职责。

1.5 脱敏日志

```

# 不记录 messages 原文，只记录数量和长度，避免日志中出现用户问题或知识库片段。
logger.info(
    "LLM call started: model=%s messages=%d prompt_chars=%d timeout=%.1f", ...
)
logger.info(
    "LLM call completed: model=%s finish_reason=%s total_tokens=%s elapsed_ms=%d", ...
)

```

解析：可观测性（model/token 用量/耗时用于计费 and 排障）与隐私合规（不记 prompt 原文）同时满足。企业项目必备，demo 一般不考虑。

1.6 三种调用形态 + 依赖注入

```

await gateway.complete(prompt, system_prompt=...) # 一次性文本
await gateway.complete_multimodal(prompt, [image_bytes], ...) # 图片转 base64 data URL 拼
content 数组
async for token in gateway.stream(prompt, ...): # 异步生成器逐 token (SSE 流式问
    答用)
    ...

```

调用方统一写法（如 rag_service.py、query_rewriter.py）：

```

def __init__(self, llm_gateway: LLMGateway | None = None, ...):
    self._llm = llm_gateway or LLMGateway(timeout=timeout)

```

解析：构造参数留注入入口 → 测试时传 mock gateway，不用 patch 框架内部。rag_engine.py 还支持按每次请求的 RAGConfig 用不同模型/超时临时构造网关。

模块二：RAG 引擎

backend/app/services/rag_engine.py

对话式问答主编排，相当于你用 LCEL 手写的那条 Chain，但这里是显式的 async 流程。

2.1 chat() 主流水线

```

async def chat(self, question, session_id, user_id, allowed_space_ids, config=None):
    # 0. 参数钳制 + 会话管理 (Redis, 30 分钟无活动过期则轮换 session_id)
    config.top_k = max(MIN_TOP_K, min(MAX_TOP_K, config.top_k)) # 1~20
    if not session_id:
        session_id = str(uuid.uuid4())
    is_expired = await self._is_session_expired(session_id)

```

```

if is_expired:
    session_id = str(uuid.uuid4())

# 1. 检索 top-K 文档块（带权限: allowed_space_ids）
chunks = await self._retrieve_chunks(question, user_id, allowed_space_ids, config.top_k)

# 2. 相似度阈值过滤（默认 0.5）
relevant_chunks = [c for c in chunks if c.score >= config.similarity_threshold]

# 3. 没有相关块 → 直接返回兜底话术（也保存会话轮次）
if not relevant_chunks:
    yield "未找到相关信息，无法回答您的问题。请尝试换一种方式提问。"
    return

# 4. 取会话历史（Redis，最近 20 轮）
history = await self._get_conversation_history(session_id)

# 5. 组装 prompt（上下文 + 历史）
prompt, system_prompt = self._build_prompt(question, relevant_chunks, history)

# 6. 流式调 LLM，边生成边 yield（配合 SSE 推到前端）
llm = self._get_llm_gateway(config)
full_response = ""
try:
    async for token in llm.stream(prompt=prompt, system_prompt=system_prompt,
                                   temperature=config.temperature,
                                   max_tokens=config.max_tokens):
        full_response += token
        yield token
except LLMGatewayError as e:
    if e.reason == "timeout": # 网关的错误码在这里被消费
        yield "服务暂时不可用，请稍后重试。"
        return
    raise

# 7. 从完整回答里解析 [1] [2] 引用
citations = self._parse_citations(full_response, relevant_chunks)

# 8. 保存本轮会话到 Redis
await self._save_turn(session_id, user_id, question, full_response, citations)

```

解析（对照你的 LangChain RAG）：

- 这就是 RetrievalQA / LCEL chain 的手写版：检索 → 拼 prompt → 生成。区别是每一步都有**生产细节**：参数钳制、会话过期轮换、相似度阈值过滤、无结果兜底、超时降级成友好话术、每步打日志。
- **第 2 步阈值过滤**是防幻觉关键：分数不够的块宁可答（返回兜底话术），也不让 LLM 硬编。
- **第 6 步**消费了模块—网关的 `reason="timeout"` 错误码——网关层和编排层的职责分界就在这里体现。
- `AsyncGenerator` 逐 token yield，配合 FastAPI 的 SSE 实现打字机效果。

2.2 `_build_prompt()` —— prompt 工程实物

```
system_prompt = (
    "你是一个企业知识库助手。请根据提供的参考资料回答用户的问题。\\n\\n"
    "规则：\\n"
    "1. 仅基于提供的参考资料回答，不要编造信息。\\n"
    "2. 在回答中使用 [1]、[2] 等格式标注引用来源，对应参考资料的编号。\\n"
    "3. 如果参考资料不足以回答问题，请明确说明。\\n"
    "4. 使用与用户问题相同的语言回答。\\n"
    "5. 回答要简洁、准确、有条理。"
)

# 上下文：每块编号 + 来源信息（文件名 > 标题链），供引用溯源
context_parts = []
for i, chunk in enumerate(chunks, start=1):
    source_info = f"[来源: {chunk.source_file}]"
    if chunk.title_chain:
        source_info += f"> {chunk.title_chain}" # 如 "第1章 > 第2节"
    source_info += "]"
    context_parts.append(f"[{i}] {source_info}\\n{chunk.content}")

# 会话历史拼进 user prompt
history_lines.append(f"用户: {content}" / f"助手: {content}")
```

解析：

- 对应 LangChain 的 `ChatPromptTemplate`，但引用机制是 **prompt 层约定** 实现的：system prompt 要求模型输出 `[1] [2]`，生成完后 `_parse_citations()` 用正则把编号映射回具体 chunk → 前端渲染可点击的引用链接。这是“回答可溯源”的完整实现方式，值得记住。
- `title_chain`（标题链）来自分块时保留的文档层级结构，让引用能定位到“某文件某章节”。

模块三：混合检索

`backend/app/services/search_service.py`

对应 LangChain 的 Retriever + Reranker，但做的是三路召回 + RRF 融合 + Cross-Encoder 精排。

3.1 `search()` 主流程

```
async def search(self, query, user_id, allowed_space_ids, page=1, page_size=10):
    # 0. 没有可访问空间 → 直接返回空（权限前置）
    if not allowed_space_ids:
        return SearchResponse(results=[], total=0, ...)

    # 1. 查询向量化：同时生成稠密向量 + 稀疏向量
    query_embedding = await self._embedding_service.embed_query(query)

    # 2. 构建两个后端的权限过滤器
    qdrant_filter = self._build_qdrant_filter(user_id, allowed_space_ids)
    opensearch_filter = self._build_opensearch_filter(user_id, allowed_space_ids)
```

```

# 3. 多路召回（带超时降级）
recall_results = await self._multi_recall(query, dense_vector, sparse_indices,
                                           sparse_values, qdrant_filter,
opensearch_filter)

# 4. RRF 融合
candidates = self._rrf_fusion(recall_results)

# 5. 只对 top-20 候选做精排（贵操作省着用），后面的保持 RRF 顺序
reranked = await self._cross_encoder_rerank(query, candidates[:RERANK_TOP_N])
all_results = reranked + candidates[RERANK_TOP_N:]

# 6. 格式化（分数归一 0-1、高亮片段 ≤200 字符）+ 分页
formatted = self._format_results(all_results, query)
return SearchResponse(results=paginated, total=total, ...)

```

解析：

- **权限过滤在检索层做**（Pre-Filtering）：把 filter 下推到 Qdrant/OpenSearch 查询里，而不是查出来再过滤——否则分页和 top_k 都会被无权文档污染。企业知识库和个人 demo 的分水岭之一。
- **稠密 + 稀疏 + BM25 三路**：稠密向量管语义（"报销流程"能命中"费用申请"），BM25 管精确关键词（型号、编号、专有名词），稀疏向量介于两者之间。

3.2 _multi_recall() —— 并发召回 + 超时降级

```

RETRIEVER_TIMEOUT = 3.0 # 单路检索器超时（秒）

tasks = [
    self._bm25_recall(query, opensearch_filter), # OpenSearch + IK 分词
    self._dense_recall(dense_vector, qdrant_filter), # Qdrant 稠密向量
    self._sparse_recall(sparse_indices, sparse_values, qdrant_filter) # Qdrant 稀疏向量
]

gathered = await asyncio.gather(
    *[asyncio.wait_for(task, timeout=RETRIEVER_TIMEOUT) for task in tasks],
    return_exceptions=True, # 关键：一路失败不拖垮其他路
)

for i, result in enumerate(gathered):
    if isinstance(result, Exception):
        logger.warning(f"{retriever_names[i]} retriever failed or timed out")
        continue # 跳过这路，用剩下的路继续
    results.append(result)

```

解析：

- `asyncio.gather` 并发三路（你的 demo 大概率是串行单次向量检索）。
- `return_exceptions=True` + **每路独立 3s 超时** = 降级设计：OpenSearch 抖动了，向量两路照样出结果，搜索不挂。和模块一 LLM 的双层超时是同一哲学：任何外部依赖都假定它会失败。
- 这个模式（并发 + 单路超时 + 失败跳过）在 `query_enhancer.py` 的查询改写/HyDE/子查询分解上也复用了。

3.3 `_rrf_fusion()` —— RRF 融合算法

```
RRF_K = 60
# RRF 公式: score(d) = Σ 1/(k + rank_i(d))

rrf_scores: dict[str, float] = {}
for results in recall_results:          # 遍历每一路的结果列表
    for rank, hit in enumerate(results, start=1):  # rank 从 1 开始
        rrf_score = 1.0 / (RRF_K + rank)
        rrf_scores[hit.chunk_id] = rrf_scores.get(hit.chunk_id, 0.0) + rrf_score

# 按 RRF 总分降序, 取 top 100 候选
sorted_chunks = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
```

解析:

- 问题: 三路结果的分数不可比 (BM25 分数、向量余弦相似度量纲不同), 不能直接相加。
- RRF 的解法: **不用原始分, 只用名次**。每路第 1 名得 1/61 分, 第 2 名得 1/62 分.....同一个 chunk 在多路出现则分数累加 → 多路都排前面的文档自然胜出。
- $k=60$ 是论文经验值, 作用是平滑: 让第 1 名和第 5 名的差距不至于碾压。
- 这段代码就 10 行, 是"多路召回如何合并"的标准答案, 面试手写题常客。

3.4 精排的供应商降级链

```
# 优先级: DashScope rerank → 本地 BGE Cross-Encoder → bigram fallback
# 中文场景 gte-rerank-v2 实测优于 bge-reranker-base, 且不占本地内存。
# 注意: DashScope rerank 不是 OpenAI 兼容协议, 所以不走 LiteLLM 而是直连 httpx。
```

解析: 精排 (rerank) 是把 query 和每个候选文档**拼在一起**过一遍 Cross-Encoder, 精度远高于双塔向量召回但贵, 所以只对 RRF 后的 top-20 做。这里又是一条"云端优先、本地兜底"的降级链, 和全项目的容错风格一致。

模块四: 入库流水线 `backend/app/tasks/pipeline.py`

对应 LangChain 的 Loader → Splitter → Embedding → VectorStore, 用 Celery chain 串成异步任务链。

4.1 任务链定义

```
pipeline = chain(
    parse_document.s(document_id),          # 1. 解析: parsers/ 插件按格式解析成中间表示
    profile_match.s(),                      # 2. Profile 匹配: 自动匹配该文档类型的解析策略
    universal_parser_check.s(),             # 3. 兜底判断: 无 Profile 时用多模态 LLM 解析
    process_document.s(),                  # 4. 清洗 + 结构识别 + 质量评分 (低分进入人工审核队列)
    chunk_document.s(),                    # 5. 分块: 按 Profile 策略, 保留标题层级
    embed_chunks.s(),                      # 6. 向量化: 稠密 + 稀疏
    index_chunks.s(),                      # 7. 双写索引: Qdrant + OpenSearch, 失败回滚
)
pipeline.apply_async()
```

每个任务的装饰器都带容错参数：

```
@_task_decorator(
    bind=True,
    max_retries=3,           # 失败自动重试 3 次
    default_retry_delay=10,  # 间隔 10 秒
    soft_time_limit=290,     # 290 秒软超时（抛异常，可清理）
    time_limit=300,         # 300 秒硬超时（强杀）
)
```

解析：

- Celery `chain`：上一个任务的返回值自动作为下一个任务的入参——和 LCEL 的 `|` 管道神似，但跑在分布式任务队列上，天然异步、可重试、可监控。
- 每个任务通过 `_update_document_status(document_id, "cleaning", ...)` 更新数据库状态 → 前端能实时看到文档处理进度；卡死的任务由 `watchdog.py` 定时捞回。
- **质量门**：第 4 步 `QualityScorer` 打分，`overall < 0.7` 进入人工审核队列（`review_queue.py`），人工修正后走 `cleanup_document_indices`（先删旧索引）→ 重新分块/向量化的回炉链路。这是"入库质量闭环"，个人 demo 完全没有的一层。

全项目反复出现的设计模式（串一遍）

模式	出现位置
超时降级（假定外部依赖会失败）	LLM 双层超时、检索单路 3s、rerank 5s、查询增强各路独立超时
多级 fallback	rerank：云端→本地→bigram；解析：插件→LLM 兜底
错误码翻译	<code>LLMGatewayError.reason</code> → 上层映射成用户友好话术
依赖注入便于 mock	所有 service 构造函数 <code>xxx=None</code> 默认自建
日志脱敏	只记长度/数量/token 数，不记内容原文
权限前置	无空间直接返回空；filter 下推到存储引擎

看源码时带着这六个模式去读，会发现处处都是它们的实例——这也是把"demo 级 RAG"和"生产级 RAG"区分的 checklist。